

Esercizi per il corso di LiP

G. Michele Pinna

Esercizio 1 Considerate l'alfabeto $\{0, 1\}$, descrivete il linguaggio in cui le parole, viste come numeri, sono multipli di 2 ed hanno almeno 3 1 consecutivi. Scrivetene una grammatica.

Esercizio 2 Considerate l'espressione regolare $(00 + 11)^*00(1 + \epsilon)$. Scrivete una grammatica che genera le parole di tale linguaggio.

Esercizio 3 Considerate il linguaggio $\{\text{fido, pimpa, fischio, piroetta, fitta}\}$. Scrivete un'espressione regolare che lo denota.

Esercizio 4 Scrivere una grammatica che genera $\{a^i b^j \mid i \leq j\}$

Esercizio 5 Scrivere una grammatica che genera $\{a^i b^j c^k \mid i \neq j \text{ oppure } j \neq k\}$

Esercizio 6 Scrivere una grammatica che genera $\{w \in \{0, 1\}^* \mid w \text{ interpretato come numero binario è divisibile per } 8\}$

Esercizio 7 Scrivere una grammatica che genera tutte le parole palindrome (cioè tali che $w = w^R$) su $\{0, 1\}$

Esercizio 8 Scrivere una grammatica che genera tutte le parole su $\{0, 1\}$ che hanno lo stesso numero di 0 e di 1

Esercizio 9 Sia G una grammatica in cui nessuna produzione è della forma $A \rightarrow \epsilon$. Mostrare che se $w \in \mathcal{L}(G)$, $|w| = n$ e ha una derivazione di m passi, allora w ha un albero sintattico di $n + m$ nodi

Esercizio 10 Considerate la grammatica $S \rightarrow 0S \mid 1S \mid 0A \mid 1A, A \rightarrow 0B, B \rightarrow 0C, C \rightarrow 0$, trovate gli alberi sintattici di 001000 e 11000

Esercizio 11 Definite un PDA per $\{a^i b^j c^k \mid i = j \text{ oppure } j = k\}$

Esercizio 12 Definite un PDA per $\{x \in \{a, b\}^* \mid \text{non esiste } w \in \{a, b\}^* \text{ tale che } ww = x\}$.

Esercizio 13 Supponete che $\mathcal{L}$ non contenga ϵ e che venga accettato da P per stack vuoto. Modificare P in modo che accetti anche ϵ per stack vuoto.

Esercizio 14 Considerate $aaaabbbbcc \in \{a^i b^j c^k \mid i = j \text{ oppure } j = k\}$. Descrivete la computazione di accettazione dell'automa che avete costruito.

Esercizio 15 Mostrate che se P è un PDA, allora esiste un PDA P' con un solo stato tale che $\mathcal{N}_{PDA}(P) = \mathcal{N}_{PDA}(P')$.

Esercizio 16 Costruire un PDA che riconosce $\{0^n 1^m \mid n \leq m \leq 2n\}$.

Esercizio 17 Costruire un PDA che riconosce $\{a^i b^j c^k \mid i = 2j \text{ oppure } j = 2k\}$.

Esercizio 18 Considerate il linguaggio $\{0^n 1^n \mid n \geq 1\}$. Trovate una grammatica in forma normale di Chomsky.

Esercizio 19 Mostrate che $\{a^i b^j c^k \mid i < j < k\}$ non è context free.

Esercizio 20 Mostrate che $\{a^n b^n c^m \mid m \leq n\}$ non è context free.

Esercizio 21 Mostrate che se $\mathcal{L}$ è context free, lo è anche $\mathcal{L}/a = \{w \mid wa \in \mathcal{L}\}$.

Esercizio 22 Considerate la sostituzione $s : \{0, 1\}^* \rightarrow \{\{w \in \{a, b\}^* \mid w = w^R\}, \{a^n b^{2n} \mid n \geq 0\}\}$. Dite quale è la grammatica di $s(\{w \in \{0, 1\}^* \mid \text{il numero di 0 in } w \text{ è il doppio del numero di 1}\})$.

Esercizio 23 Mostrate che $\{0^n 1^{2n} \mid n \text{ pari}\}$ è context free.

Esercizio 24 Considerate la grammatica $S \rightarrow B, B \rightarrow aBc \mid C, C \rightarrow bCc \mid \epsilon$. Trovate il FIRST ed il FOLLOW di ogni simbolo.

Esercizio 25 Fate il parsing top down delle seguenti parole aabbcccc, aabbbcccc e abbbbcccc dicendo dove eventualmente fallisce.

Esercizio 26 Data la grammatica $S \rightarrow E, E \rightarrow E + T \mid T, T \rightarrow n \mid (E)$, dite quali sono gli item LR(0) e costruite l'automa per riconoscere le maniglie. Non avete bisogno di aumentare la grammatica (è già aumentata)

Esercizio 27 Costruite le tabelle **action** e **goto** per la grammatica vista prima, usando il FOLLOW.

Esercizio 28 Fate il parsing bottom up di $x + (y + x)$ (x e y sono generate da n).

Esercizio 29 Valutate i seguenti termini, assumendo che lo scope sia dinamico, usando la relazione $\rightarrow$:

1. $t_1 = \text{sum}(\text{apply}(\text{fun}(y, \text{sum}(y, 1)), 4), \text{apply}(\text{fun}(x, x), 2))$
2. $t_2 = \text{let } f = \text{fun}(x, \text{if equal}(x, 0) \text{ then } 1 \text{ else } \text{sum}(x, x)) \text{ in } \text{apply}(f, 2)$
3. $t_3 = \text{apply}(\text{fun}(x, x), \text{fun}(z, \text{sum}(3, z)))$
4. $t_4 = \text{sum}(\text{apply}(\text{fun}(x, \text{apply}(x, 4)), \text{fun}(z, \text{sum}(3, z))), 3)$
5. $t_5 = \text{sum}(\text{apply}(\text{fun}(x, \text{sum}(x, 3)), 5), \text{apply}(\text{fun}(g, y, \text{sum}(4, y)), 1)$
6. $t_6 = \text{minus}(\text{apply}(\text{fun}(x, \text{sum}(x, x)), 8), \text{apply}(\text{fun}(x, \text{sum}(x, 5)), 1))$
7. $t_7 = \text{apply}(\text{fun}(x, \text{apply}(x, \text{apply}(\text{fun}(y, \text{sum}(y, 2)), 1))), \text{fun}(z, \text{sum}(3, z)))$
8. $t_8 = \text{fun}(x, \text{if equal}(x, y) \text{ then } x \text{ else } y)$
9. $t_9 = \text{let } y = t_2 \text{ in let } f = t_8 \text{ in equal}(\text{apply}(f, t_3), t_6)$

Esercizio 30 Descrivete, per ogni termine del precedente esercizio, tutti gli ambienti, usando la relazione $\rightsquigarrow$.

Esercizio 31 Valutate i termini dell'esercizio precedente assumendo che lo scope sia statico usando le due relazioni semantiche date.

Esercizio 32 Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '*' indica la moltiplicazione tra interi. Se le funzioni restituiscono un valore, che può anche essere una funzione, il valore restituito è indicato dalla keyword **return**. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate $f(x) + g(y)$ può accadere che i side effects dell'esecuzione della $f(x)$ si ripercuotano sull'esecuzione della $g(y)$ che avviene dopo l'esecuzione della $f(x)$.

```
01 {int x = 1;
02   int y = 2;
03   int z = 3;
04   int f (int y) {
05     y = x * z;
06     return y};
07   int m (int x) {;
08     return y + x};
09   int h1 (int w) {
10     int x = 3;
11     x = f(x) * w;
12     return x};
13   int h2 (int x) {
14     int w = 1;
15     w = f(x) + w;
16     return w};
17   int g (int n) {
18     int x = 2;
19     int z = 3;
20     x = h1(x);
21     z = h2(z);
22     y = m(x) + f(n) + z;
23     return y};
24   {int x = 1;
25     int z = 1;
26     x = g(x) + h2(y) + y;}}
```

1. che valore contiene la variabile x alla linea 26 se lo scope è statico
2. che valore contiene la variabile x alla linea 26 se lo scope è statico e i parametri delle funzioni sono tutti passati per riferimento
3. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
4. che valore contiene la variabile x alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e i parametri delle funzioni sono tutti passati per riferimento
5. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding e i parametri delle funzioni sono tutti passati per riferimento

Esercizio 33 Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '*' indica la moltiplicazione tra interi. Se le funzioni restituiscono un valore, che può anche essere una funzione, il valore restituito è indicato dalla keyword **return**. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate $f(x) + g(y)$ può accadere che i side effects dell'esecuzione della $f(x)$ si ripercuotano sull'esecuzione della $g(y)$ che avviene dopo l'esecuzione della $f(x)$.

```

01 {int x = 1;
02   int y = 2;
03   int z = 3;
04   int f (int y) {
05     y = x * z;
06     return y};
07   int d (int x) {;
08     return f(y) + x};
09   int h (int w) {
10     int m (int x) {
11       int w = 1;
12       w = f(x) + w;
13       return w};
14     w = m(x) + w;
15     return w};
16   int g (int n) {
17     int x = 3;
18     int z = 2;
19     x = h(x);
20     y = d(n) + f(z) + z;
21     return y};
22   int k (int m) {
23     return g(m)};
24   {int x = 2;
25     int z = 1;
26     x = k(x) + h(y) + y;}}

```

1. che valore contiene la variabile x alla linea 26 se lo scope è statico
2. che valore contiene la variabile x alla linea 26 se lo scope è statico e i parametri delle funzioni sono tutti passati per riferimento
3. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
4. che valore contiene la variabile x alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e i parametri delle funzioni sono tutti passati per riferimento
5. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding e i parametri delle funzioni sono tutti passati per riferimento

Esercizio 34 Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il ‘*’ indica la moltiplicazione tra interi. Se le funzioni restituiscono un valore, che può anche essere una funzione, il valore restituito è indicato dalla keyword **return**. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate $f(x) + g(y)$ può accadere che i side effects dell’esecuzione della $f(x)$ si ripercuotano sull’esecuzione della $g(y)$ che avviene dopo l’esecuzione della $f(x)$.

```

01 {int x = 1;
02   int y = 2;
03   int z = 3;
04   int f (int y) {
05     y = x * z;
06     return y};
07   int d (int x) {;
08     return f(y) + x};
09   int h (int -> int o) {
10     int m (int x) {
11       int w = 1;
12       x = f(x) + w;
13       return w + x};
14     int x = 2;
15     z = m(x) + o(z) + x;
16     return z};
17   int g (int n) {
18     int x = 3;
19     int z = 2;
20     x = f(x);
21     y = d(n) + f(z) + z;
22     return y};
23   int k (int n) {
24     return g(n)};
25   {int x = 4;
26     int z = 1;
27     x = k(x) + h(f) + y;}}

```

1. che valore contiene la variabile x alla linea 26 se lo scope è statico
2. che valore contiene la variabile x alla linea 26 se lo scope è statico e i parametri delle funzioni sono tutti passati per riferimento
3. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
4. che valore contiene la variabile x alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e i parametri delle funzioni sono tutti passati per riferimento
5. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding e i parametri delle funzioni sono tutti passati per riferimento

Esercizio 35 Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '*' indica la moltiplicazione tra interi. Se le funzioni restituiscono un valore, che può anche essere una funzione, il valore restituito è indicato dalla keyword **return**. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate $f(x) + g(y)$ può accadere che i side effects dell'esecuzione della $f(x)$ si ripercuotano sull'esecuzione della $g(y)$ che avviene dopo l'esecuzione della $f(x)$.

```

01 {int x = 1;
02   int y = 2;
03   int z = 3;
04   int f (int y) {
05     y = x * z;
06     return y};
07   int d (int x) {;
08     return y + x};
09   int h (int -> int o) {
10     int m (int x) {
11       int w = 1;
12       w = x + w;
13       return w};
14   int x = 2;
15     z = m(x) + o(z) + x;
16     return z};
17   int g (int n) {
18     int x = 3;
19     int z = 2;
20     x = f(x);
21     y = d(n) + f(z) + n;
22     return y};
23   int k (int n) {
24     return g(n) + n};
25   {int x = 2;
26     int z = 2;
27     x = k(x) + h(f) + y;}}

```

1. che valore contiene la variabile x alla linea 26 se lo scope è statico
2. che valore contiene la variabile x alla linea 26 se lo scope è statico e i parametri delle funzioni sono tutti passati per riferimento
3. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
4. che valore contiene la variabile x alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e i parametri delle funzioni sono tutti passati per riferimento
5. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding e i parametri delle funzioni sono tutti passati per riferimento

Esercizio 36 Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '*' indica la moltiplicazione tra interi. Se le funzioni restituiscono un valore, che può anche essere una funzione, il valore restituito è indicato dalla keyword **return**. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate $f(x) + g(y)$ può accadere che i side effects dell'esecuzione della $f(x)$ si ripercuotano sull'esecuzione della $g(y)$ che avviene dopo l'esecuzione della $f(x)$.

```

01 {int x = 1;
02  int y = 2;
03  int z = 3;
04  int f (int y) {
05      y = x * z;
06      return y};
07  int d (int x) {;
08      return y + x};
09  int h (int -> int o) {
10      int m (int x) {
11          int w = 1;
12          z = x + w;
13          return w};
14  int x = 2;
15      z = m(x) + o(z) + x;
16      return z};
17  int g (int n) {
18      int x = 3;
19      int y = 2;
20      x = f(x);
21      y = d(n) + f(n) + x;
22      return y};
23  int k(int n) {
24      return g(n) + n};
25  {int x = 1;
26      int z = 1;
27      x = k(x) + h(f) + y;}}
```

1. che valore contiene la variabile x alla linea 26 se lo scope è statico
2. che valore contiene la variabile x alla linea 26 se lo scope è statico e i parametri delle funzioni sono tutti passati per riferimento
3. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
4. che valore contiene la variabile x alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e i parametri delle funzioni sono tutti passati per riferimento
5. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding e i parametri delle funzioni sono tutti passati per riferimento

Esercizio 37 Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '*' indica la moltiplicazione tra interi. Se le funzioni restituiscono un valore, che può anche essere una funzione, il valore restituito è indicato dalla keyword **return**. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate $f(x) + g(y)$ può accadere che i side effects dell'esecuzione della $f(x)$ si ripercuotano sull'esecuzione della $g(y)$ che avviene dopo l'esecuzione della $f(x)$.

```

01 {int x = 1;
02  int y = 2;
15      z = m(x) + o(z) + x;
16      return z};
```

```

03  int z = 3;
04  int f (int y) {
05      y = x * z;
06      return y;
07  int d (int x) {;
08      return y + x;
09  int h (int -> int o) {
10      int m (int x) {
11          int w = 1;
12          x = x + w;
13          return w;
14      int x = 2;

17  int g (int n) {
18      int x = 3;
19      int y = 2;
20      x = f(x);
21      z = d(n) + f(n) + x;
22      return z + x;
23  int k (int n) {
24      return g(n) + n;
25  {int x = 2;
26      int z = 1;
27      x = k(x) + h(f) + z;}}

```

1. che valore contiene la variabile x alla linea 26 se lo scope è statico
2. che valore contiene la variabile x alla linea 26 se lo scope è statico e i parametri delle funzioni sono tutti passati per riferimento
3. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
4. che valore contiene la variabile x alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e i parametri delle funzioni sono tutti passati per riferimento
5. che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding e i parametri delle funzioni sono tutti passati per riferimento

Esercizio 38 Considerate il seguente tipo

$$((\text{int} \rightarrow \text{bool}) * \text{int} \rightarrow \text{bool}) \rightarrow (\text{bool} \rightarrow \text{int})$$

scrivete un termine in Ocaml-like che ha tale tipo e derivatelo. Dovete indicare quale Γ usate, quindi dovete specificare per ogni variabile che compare nel termine, quale tipo è associato.

Esercizio 39 Considerate il seguente tipo

$$(\text{int} * (\text{int} \rightarrow \text{bool}) \rightarrow \text{int}) \rightarrow ((\text{int} \rightarrow \text{int}) \rightarrow \text{bool})$$

scrivete un termine in Ocaml-like che ha tale tipo e derivatelo. Dovete indicare quale Γ usate, quindi dovete specificare per ogni variabile che compare nel termine, quale tipo è associato.

Esercizio 40 Considerate il seguente tipo

$$(\text{int} \rightarrow \text{bool}) \rightarrow ((\text{int} \rightarrow \text{bool}) * \text{bool} \rightarrow (\text{int} \rightarrow \text{bool})) \rightarrow (\text{int} \rightarrow \text{bool})$$

scrivete un termine in Ocaml-like che ha tale tipo e derivatelo. Dovete indicare quale Γ usate, quindi dovete specificare per ogni variabile che compare nel termine, quale tipo è associato.

Linguaggio Fun e le relazioni ‘ $\rightarrow$ ’ e ‘ $\sim$ ’.

I termini del linguaggio sono generati dalla seguente grammatica.

$$t ::= c \mid x \mid \text{op}_i(t_1, \dots, t_{a_i}) \mid \text{if}(t_1, t_2, t_3) \mid \text{fun}(x, t) \mid \text{apply}(t_1, t_2) \mid \text{let } x = t_1 \text{ in } t_2$$

$$\begin{array}{c}
\overline{\langle x, \delta \rangle \rightarrow \delta(x)} \qquad \qquad \qquad \overline{\langle c, \delta \rangle \rightarrow c} \qquad \qquad \qquad \overline{\langle \text{fun}(x, t), \delta \rangle \rightarrow \text{fun}(x, t)} \\
\\
\frac{\langle t_1, \delta \rangle \rightarrow v_1 \quad \dots \quad \langle t_{a_i}, \delta \rangle \rightarrow v_{a_i}}{\langle \text{op}_i(t_1, \dots, t_{a_i}), \delta \rangle \rightarrow \widetilde{\text{op}_i}(v_1, \dots, v_{a_i})} \qquad \frac{\langle t_0, \delta \rangle \rightarrow \text{true} \quad \langle t_1, \delta \rangle \rightarrow v}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow v} \\
\\
\frac{\langle t_0, \delta \rangle \rightarrow \text{false} \quad \langle t_2, \delta \rangle \rightarrow v}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow v} \qquad \frac{\langle t_1, \delta \rangle \rightarrow \text{fun}(x, t) \quad \langle t_2, \delta \rangle \rightarrow v' \quad \langle t, \delta[x/v_2] \rangle \rightarrow v}{\langle \text{apply}(t_1, t_2), \delta \rangle \rightarrow v} \\
\\
\frac{\langle t_1, \delta \rangle \rightarrow v_1 \quad \langle t_2, \delta[x/v_1] \rangle \rightarrow v}{\langle \text{let } x = t_1 \text{ in } t_2, \delta \rangle \rightarrow v}
\end{array}$$

Figura 1: La relazione big-step ‘ $\rightarrow$ ’

$$\begin{array}{c}
\overline{\langle\langle x, \Delta \rangle\rangle \rightsquigarrow \langle\text{top}(\Delta)(x), \Delta\rangle} \qquad \qquad \qquad \overline{\langle\langle \text{op}_i(v_1, \dots, v_{a_i}), \Delta \rangle\rangle \rightsquigarrow \langle\langle v, \Delta \rangle\rangle} \\
\\
\frac{\langle\langle t_j, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_j, \Delta' \rangle\rangle}{\overline{\langle\langle \text{op}_i(v_1, \dots, v_{j-1}, t_j, \dots, t_{a_i}), \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{op}_i(v_1, \dots, v_{j-1}, t'_j, \dots, t_{a_i}), \Delta' \rangle\rangle}} \\
\\
\overline{\langle\langle \text{if true then } t_1 \text{ else } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle t_1, \Delta \rangle\rangle} \qquad \overline{\langle\langle \text{if false then } t_1 \text{ else } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle t_2, \Delta \rangle\rangle} \\
\\
\frac{\langle\langle t_0, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_0, \Delta' \rangle\rangle}{\overline{\langle\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{if } t'_0 \text{ then } t_1 \text{ else } t_2, \Delta' \rangle\rangle}} \\
\\
\overline{\langle\langle \text{apply}(\text{fun}(x, t), v), \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{funblock}(t), \text{push}(\text{top}(\Delta)[x/v], \Delta) \rangle\rangle} \\
\\
\frac{\langle\langle t_1, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_1, \Delta' \rangle\rangle}{\overline{\langle\langle \text{apply}(t_1, t_2), \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{apply}(t'_1, t_2), \Delta' \rangle\rangle}} \\
\\
\frac{\langle\langle t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_2, \Delta' \rangle\rangle}{\overline{\langle\langle \text{apply}(\text{fun}(x, t), t_2), \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{apply}(\text{fun}(x, t), t'_2), \Delta' \rangle\rangle}} \\
\\
\overline{\langle\langle \text{funblock}(t), \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{funblock}(t'), \Delta' \rangle\rangle} \qquad \overline{\langle\langle \text{funblock}(v), \Delta \rangle\rangle \rightsquigarrow \langle\langle v, \text{pop}(\Delta) \rangle\rangle} \\
\\
\frac{\langle\langle t_1, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_1, \Delta' \rangle\rangle}{\overline{\langle\langle \text{let } x = t_1 \text{ in } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{let } x = t'_1 \text{ in } t_2, \Delta' \rangle\rangle}} \\
\\
\overline{\langle\langle \text{let } x = v \text{ in } t, \Delta \rangle\rangle \rightsquigarrow \langle\langle \text{funblock}(t), \text{push}(\text{top}(\Delta)[x/v], \Delta) \rangle\rangle}
\end{array}$$

Figura 2: La relazione small-step ‘ $\rightsquigarrow$ ’

Un linguaggio funzionale e le regole per l'inferenza di tipo

Il linguaggio considerato è generato dalla seguente grammatica:

$$t ::= x \mid n \mid true \mid false \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t_1 \mid t_1 = t_2 \mid t_1 < t_2 \mid \\ if\ t_0\ then\ t_1\ else\ t_2 \mid fun(x, t) \mid apply(t_1, t_2) \mid pair(t_1, t_2) \mid fst(t) \mid snd(t)$$

mentre i tipi sono

$$\tau ::= a \mid int \mid bool \mid \tau_1 \rightarrow \tau_2 \mid \tau_1 * \tau_2$$

dove a sono le variabili di tipo. Infine Γ è un insieme di coppie (x, τ) dove x è una variabile del linguaggio e τ un tipo.

$\frac{(x, \tau) \in \Gamma}{\Gamma \vdash x : \tau}$	$\overline{\Gamma \vdash n : int}$	$\overline{\Gamma \vdash true : bool}$	$\overline{\Gamma \vdash false : bool}$
$\frac{\Gamma \vdash t_1 : int \quad \Gamma \vdash t_2 : int}{\Gamma \vdash t_1\ op\ t_2 : int} \quad op \in \{+, -, \times\}$	$\frac{\Gamma \vdash t_1 : \tau \quad \Gamma \vdash t_2 : \tau}{\Gamma \vdash t_1 = t_2 : bool} \quad \tau \neq \tau_1 \rightarrow \tau_2$		
$\frac{\Gamma \vdash t_1 : bool \quad \Gamma \vdash t_2 : bool}{\Gamma \vdash t_1\ op\ t_2 : bool} \quad op \in \{\wedge, \vee\}$	$\frac{\Gamma \vdash t : bool}{\Gamma \vdash \neg t : bool}$	$\frac{\Gamma \vdash t_1 : int \quad \Gamma \vdash t_2 : int}{\Gamma \vdash t_1 < t_2 : bool}$	
$\frac{\Gamma \vdash x : \tau_1 \quad \Gamma \vdash t : \tau_2}{\Gamma \vdash fun(x, t) : \tau_1 \rightarrow \tau_2}$	$\frac{\Gamma \vdash t_0 : bool \quad \Gamma \vdash t_1 : \tau \quad \Gamma \vdash t_2 : \tau}{\Gamma \vdash if\ t_0\ then\ t_1\ else\ t_2 : \tau}$	$\frac{\Gamma \vdash t_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash t_2 : \tau_1}{\Gamma \vdash apply(t_1, t_2) : \tau_2}$	
$\frac{\Gamma \vdash t_1 : \tau_1 \quad \Gamma \vdash t_2 : \tau_2}{\Gamma \vdash pair(t_1, t_2) : \tau_1 * \tau_2}$	$\frac{\Gamma \vdash t : \tau_1 * \tau_2}{\Gamma \vdash fst(t) : \tau_1}$	$\frac{\Gamma \vdash t : \tau_1 * \tau_2}{\Gamma \vdash snd(t) : \tau_2}$	

Figura 3: La relazione ‘ $\vdash$ ’ per l’inferenza di tipo